

JEE-T : Automated Practice Paper Generator System for the JEE

Arunim Gupta
arunim19025@iiitd.ac.in
IIIT Delhi
New delhi, Delhi, India

Samiksha Modi
samiksha19331@iiitd.ac.in
IIIT Delhi
New Delhi, Delhi, India

Arshad Abbas Shahabuddin
arshad19300@iiitd.ac.in
IIIT Delhi
New delhi, Delhi, India

Shabeg Singh Gill
shabeg19388@iiitd.ac.in
IIIT Delhi
New Delhi, Delhi, India

Aryan GD Singh
aryan19459@iiitd.ac.in
IIIT Delhi
New delhi, Delhi, India

Yash Mathne
yash19129@iiitd.ac.in
IIIT Delhi
New Delhi, Delhi, India

ACM Reference Format:

Arunim Gupta, Arshad Abbas Shahabuddin, Aryan GD Singh, Samiksha Modi, Shabeg Singh Gill, and Yash Mathne. 2018. JEE-T : Automated Practice Paper Generator System for the JEE. In *Proceedings of ACM Conference (Conference'17)*. ACM, New York, NY, USA, 3 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 ABSTRACT

In this paper, we aim to propose a system to help students in their preparation for the JEE. We use adaptive techniques to ensure the learning is best suited to the individual by assessing their strengths and weaknesses when they use our application to practice questions.

2 INTRODUCTION

JEE aspirants waste a lot of time deciding which questions to solve from the vast number of resources they use for preparation. It is not feasible to solve all of them. Since they pick the questions unsystematically, they may not be able to cover all the concepts from a particular chapter. As a result, they repeatedly practice questions pertaining to the same concepts.

To solve this problem, we aim to create a user-friendly application using streamlit that would provide JEE aspirants with practice questions that are curated for them based on their knowledge level that we evaluate and update after every attempt. We also ensure that every chapter is thoroughly covered at a sub-topic level and avoid repetition of problems. Additionally, the student will be able to see his performance in different chapters, subjects and analyze his weak and strong areas on the dashboard. The novelty lies in how the question's difficulty is assessed and creating the users knowledge level. We use this data to give the user more questions from the weak areas and fewer from the strong areas in future practice papers to help them keep in touch with the concepts.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2018 Association for Computing Machinery.
ACM ISBN 978-1-4503-XXXX-X/18/06...\$15.00
<https://doi.org/XXXXXXX.XXXXXXX>

On top of that, it is always challenging to find relevant information to fill the knowledge gaps exposed during testing. So we also have a recommendation system that suggests the most relevant documents for the question.

For this project, we created our own dataset of 1800 single correct MCQ Questions and True/False Questions from past JEE archives in English. It has the following features - QID (Question ID), domain, subdomain, question, imageURL, options, answer and explanation. This problem is widespread among JEE aspirants every year. We also experienced it while preparing for the JEE and wish to make things easier for future JEE aspirants.

3 LITERATURE REVIEW

The mechanism of generating questions papers in an automated manner is based upon various methods and techniques. Various aspects of the same have been presented by authors in their papers.

3.1 Automated Test Paper Generation Using Utility Based Agent and Shuffling Algorithm

3.1.1 Methodology. In [1], shuffling algorithm is applied to design different sets of the same paper without any repetition or duplication. Old questions were flagged to avoid redundancy.

Along with this, a 'knowledge base' was maintained, combining both the utility based agent and a shuffling algorithm to randomize the selection of questions. The system involves the examiner logging into the system, specifying the question type, the difficulty level and other attributes like course and topic. Overall difficulty of the paper was also computed. The technology used was MySQL, PHP and Javascript.

3.1.2 Limitation. There are a few pitfalls in the system. The system is institute specific and hence cannot be generalized. The number of questions cannot be less than 20 and is restricted to only the English language.

3.2 Automatic Question Paper Generator System

3.2.1 Methodology. Paper [2] helped us form a basis of our project by providing relevant information about what we would need. It provided a baseline design for the application and its various modules. It proposed the non repetition of questions and the mapping

of the outcomes that the users would get from using the application, and also included chapter-wise question generation as well.

3.3.2 Limitation. The random algorithm can lead to some problems. Too much randomisation could lead to creation of mediocre question papers. Also, non-categorisation of question levels makes it difficult to use the application.

3.3 Automatic Generation of Question Paper from User Entered Specifications Using a Semantically Tagged Question Repository

3.3.1 Methodology. Paper [3] used a semantically tagged dataset for question paper generation. It tagged questions into different types. Question tags were assigned values and upper and lower bounds were used during search. The Search Engine randomly picked questions satisfying the given criteria. Finally, it was implemented as a web application for simplicity.

3.3.2 Limitation. Using untagged question datasets is difficult as tagging takes a lot of effort. The requirement of user specifications every time is inconvenient.

3.4 Developing an intelligent agent for Automatic Question Paper setting

3.4.1 Methodology. Paper [4] proposed the use of an "Automated Question Paper Generation Agent" (AQPA) for generating question papers. The agent used backtracking and random number generation algorithms to generate the papers. For the database, the system used a hash-table database to reduce space and optimise time. The model also used a question paper generation module to generate question papers according to a particular format. The module contained pre-defined formats that could be used to design the paper. It also made sure the questions from the dataset were not repeated within the same question paper.

3.4.2 Limitation. The AQPA does not keep track of the questions attempted by the user. This could cause questions to repeat in future papers and hence, the random number generation would not assure the best choice of questions from the question bank. The system could have possibly included a way to include topic wise question paper generation.

3.5 Smart Question (SQ)

3.5.1 Methodology. In [5] a concept of non-repeating questions was proposed using Smart Questions. A smart question is a concept that involves converting a static question with set numbers and answer options into a dynamic question with parameters. The response options are unsorted, and the parameters list potential values. The smart question idea made it easier to generate many comparable questions from a single question template while also saving a lot of space in a database.

3.5.2 Limitation. Although smart questions are a strong concept, implementation of the same on the dynamic dataset is challenging, especially if the database is expansive. According to the current technology, it requires manual intervention to depict the semantics of the question. Since the software is in its infancy, it is only limited to objective choices.

3.6 Automatic Generation of Question Bank Based on Predefined Templates

3.6.1 Methodology. Paper [7] proposed a concept of non-repeating questions by using Predefined Templates. A Predefined template, as suggested, uses questions or knowledge banks stored in predefined formats to generate multiple types of questions. The questions generated can be of various types which are predefined.

3.6.2 Limitation. The system architecture development is a complex and expensive task. Besides, not every question can be converted into the proposed predefined template. This limits the scope of the implementation of the architecture.

4 METHODOLOGY

4.1 Dataset Creation

First we created a repository of questions by scraping them from publicly available sources. This was done using Beautiful Soup. Our dataset consists of the following 8 features - QID, domain, subdomain, question, imageURL, options, answer and explanation, and has a total of 1806 samples. We employed Naive Bayes Classifier for classifying questions into their respective domains and subdomains.

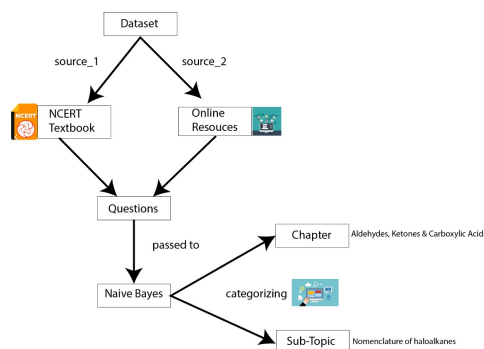


Figure 1: Handling the Data

Now, we used the NCERT textbook as the corpus and used ranking methods such as BM25 and Cosine Similarity to get the top ranking subdomains for the given question. In this manner, we are using an search pipeline to classify our questions.

4.2 Diagnostic Test

To ensure a user tailored experience, we use a diagnostic test to create a knowledge map of the user. The diagnostic test tries to cover a wide range of domains and subdomains. The results of this test are used to create a subdomain level knowledge graph wherein each subdomain is categorised into easy, medium and hard. This functionality would be user oriented.

Then, we will develop a model for question paper generation. The

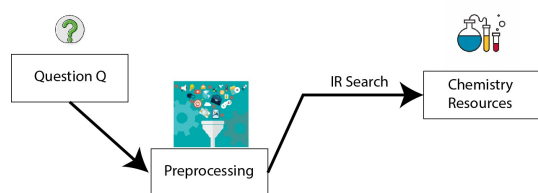


Figure 2: Document Search for Relevant Documents

model will alleviate 3 difficulties faced by students - repetitive questions, questions that don't focus on their weak topics and being able to understand their strengths and weaknesses at a sub-domain level.

Our model deals with repetitive questions by using special data structures for the same. When a user first uses the application, the user is presented with a diagnostic test specially designed to create a knowledge map of the user. We do this by analysing their performance at a sub-domain level and setting a time threshold to classify sub domains and domains into easy, medium or hard.

Then, we use this knowledge graph to generate practice test papers reflecting the same by means of number of questions per domain. The knowledge map is continuously updated based upon the performances in the practice tests.

4.3 Recommended Material

If a user gets a question incorrectly, we suggest relevant material to fill the knowledge gap. To do so, we use techniques such as BM25 and Cosine Similarity to rank the documents. To get the relevance scores for the same, we use a ground truth model that searches for the subdomain data of the question with our resources database. We assume since we get the documents in our resource closest to the subdomain in which this question belongs, this is true relevance data.

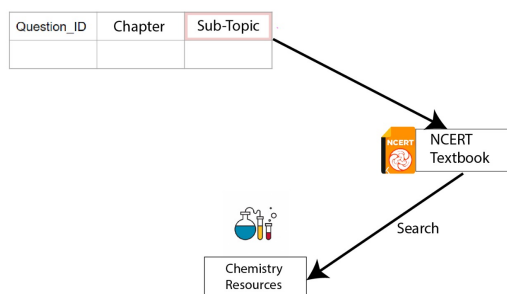


Figure 3: Ground Truth

5 EVALUATION

5.1 System

For evaluating the results obtained using our ranking functions, we are using and human supervised results from the ground truth

system that we have created.

Since the ground truth system isn't 100% accurate, we used the results from the ground truth system and used domain knowledge within the to run tests for 200 questions.

5.2 Results Analysis

The results that we got are as follows:

Method	BM25	Cosine Sim.	Naive Bayes
MAP	63.07	56.12	39.97
MAR	61.88	49.96	31.45
nDCG	68.23	63.44	40.20

As a result, we use the BM25 as the ranking metric.

5.3 Limitations

At the moment, our system is limited to only Chemistry and it's respective domains and sub-domains. Also, our dataset is not so exhaustive and can be extended further.

5.4 Challenges

Crowd Sourcing high quality data required a lot of time and manpower. Finding links that directly correspond to questions where the user has answered incorrectly was a cumbersome task.

5.5 Future Work

At the moment, we have included only Chemistry due to time constraints and limited manpower. In the future, we aim to extend our work by including more subjects and their corresponding domains and subdomains.

6 CONCLUSION

We have successfully created a platform that alleviates the common problem faced by the users. We have used information retrieval and machine learning techniques to either automate manual input or suggest relevant study material. In doing so we have provided a proof of concept system of the novel concept we suggested.

REFERENCES

- [1] 'El-Rahman, S. and Zolait, A. 2019. Automated Test Paper Generation Using Utility Based Agent and Shuffling Algorithm. International Journal of Web-Based Learning and Teaching Technologies 14, 1, 69-83.
- [2] Kiran, F., Gopal, H. and Dalvi, A. 2017. Automatic Question Paper Generator System. International Journal of Computer Applications 166, 10, 42-47.
- [3] Nalawade, G. and Ramesh, R. 2016. Automatic Generation of Question Paper from User Entered Specifications Using a Semantically Tagged Question Repository. 2016 IEEE Eighth International Conference on Technology for Education (T4E).
- [4] Pinjani, K., Raut, R. and Yedey, S., 2015. Developing an intelligent agent for Automatic Question Paper setting. [online] Ijecsce.org. Available at: <<https://ijecsce.org/papers/ATCON2015/HPC-09.pdf>> [Accessed 26 February 2022].
- [5] Ugurdag, H., Argali, E., Eker, O. and Basaran, A., 2009. Smart Question (sQ): Tool for Generating Multiple-Choice Test Questions. [online] Available at: <https://www.researchgate.net/publication/229005305_Smart_Question_sQ_Tool_for_Generating_Multiple-Choice_Test_Questions> [Accessed 26 February 2022].
- [6] Ezz, A. and Dahab, M. 2014. Automatic Generation of Question Bank Based on Pre-defined Templates. International Journal of Innovations Advancement in Computer Science 3, 1, 80-87.
- [7] <https://www.ncertbooks.guru/ncert-books-class-12-chemistry/>